

ICDFA PRACTICAL LAB REPORT

Course: Intro to Critical Infrastructure for IT Learners

Lab Assignment: LAB 01 — Introduction to Programmable Logic Controllers

Student Name: JOSEPH AGYAPONG

Student ID: C1126ACIS17354

Submission Date: September 19, 2026

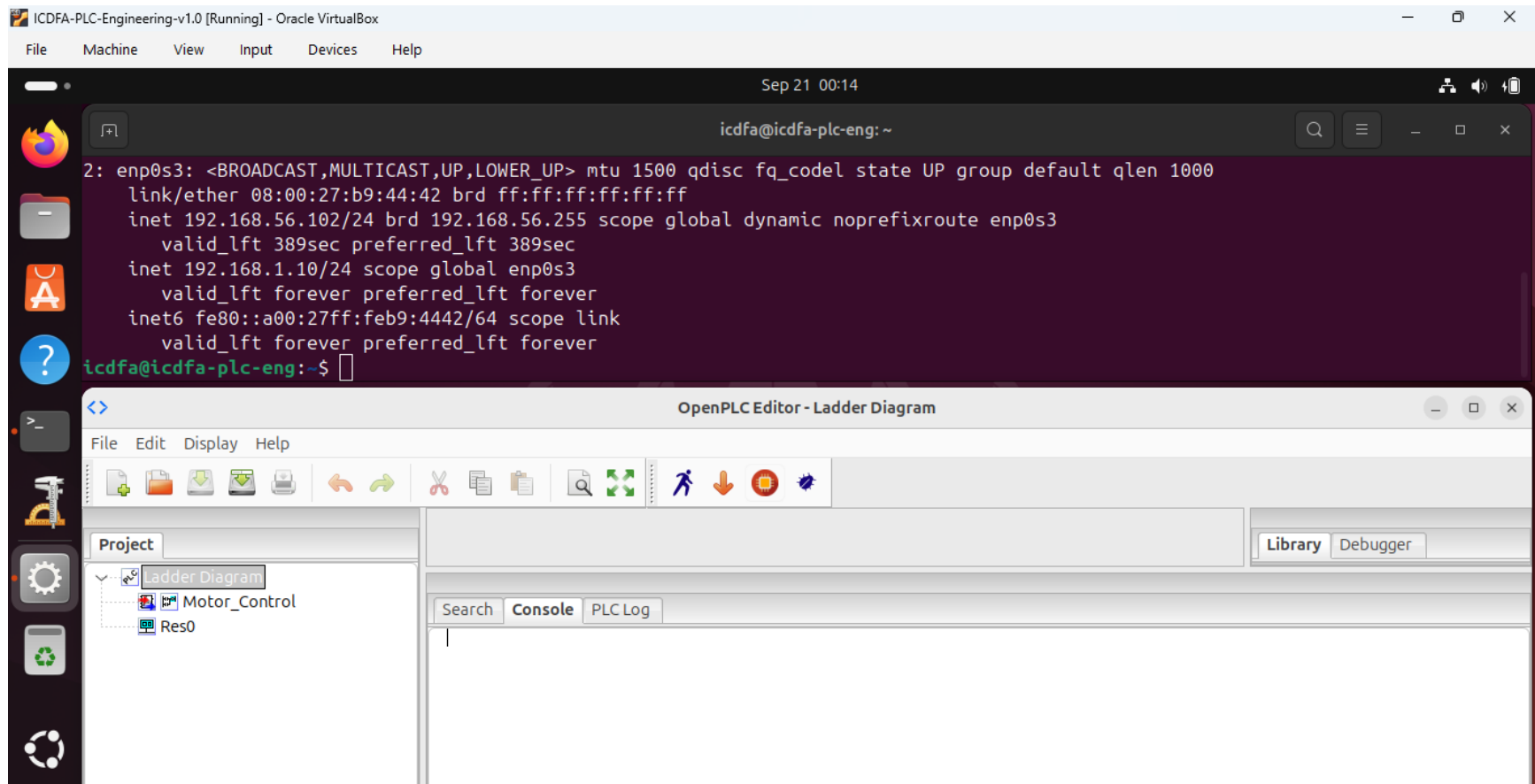
Evidence 1 — Laboratory Environment and PLC Identification

1. Target System Configuration & Environment Details

To meet the laboratory access criteria, the ICDFA PLC Virtual Machine was deployed in an isolated network environment.

- **Virtual Machine Name:** ICDFA-PLC-Engineering-v1.0
- **Operating System:** Ubuntu Linux (Custom ICDFA Industrial Workstation Edition)
- **Verified IP Address:** 192.168.1.10/24 (*Interface enp0s3*)
- **User Account:** icdfa
- **Network Mode:** Host-Only / Internal Isolated Network
- **Primary Application:** OpenPLC Editor

2. Screenshot 1 — Operational PLC Environment

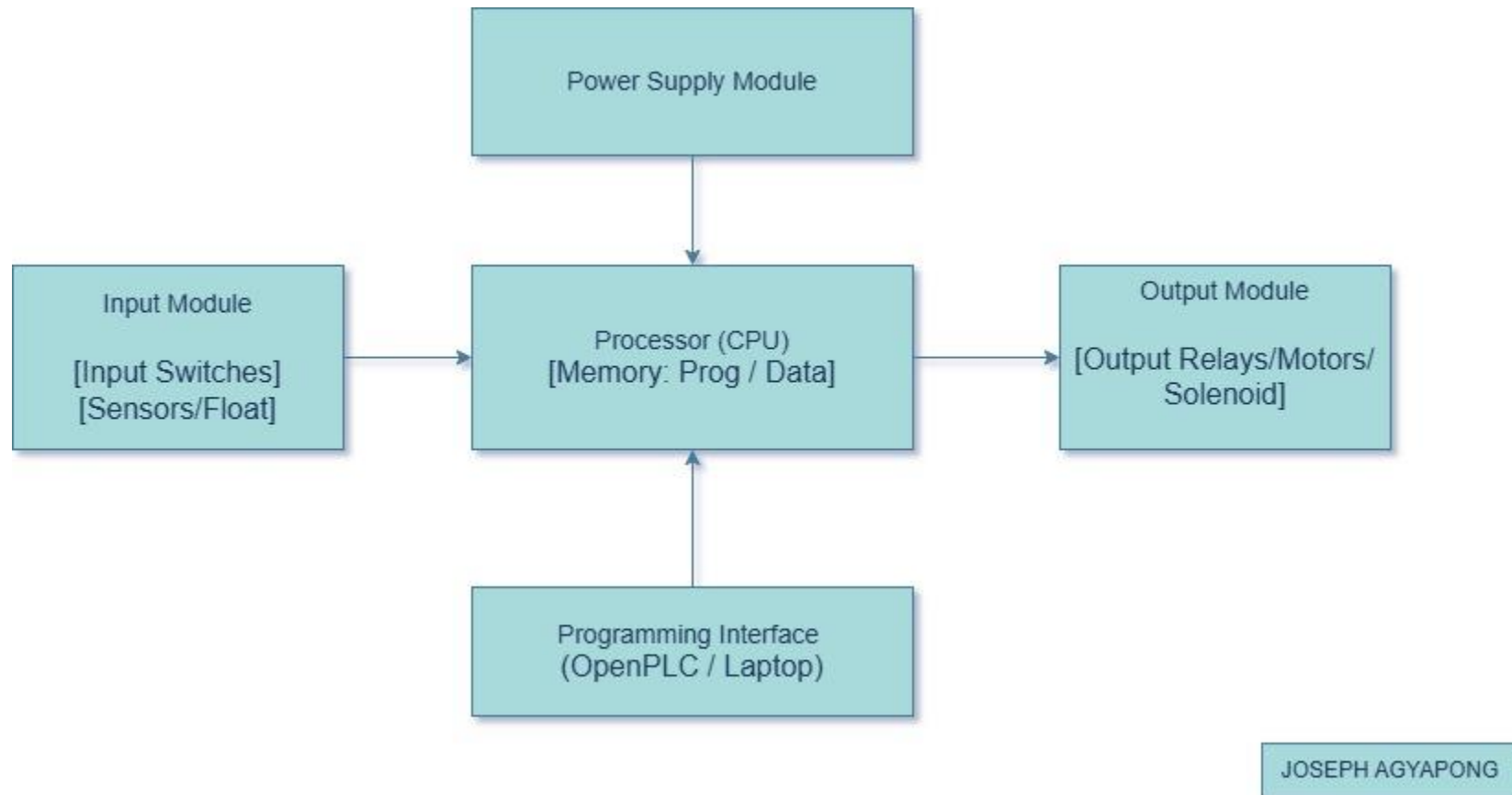


Caption: Figure 1.1 — Operational ICDFA PLC Virtual Machine desktop displaying terminal IP verification (192.168.1.10/24) alongside the launched OpenPLC Editor application.

Evidence 2 — PLC Architecture Analysis

1. Overview of PLC Architecture

A Programmable Logic Controller, PLC, is an industrial microcomputer used to perform control functions in industrial control systems. Unlike ordinary microcomputers, the PLC is modular and interfaces directly with sensors, actuators, and with circuitry which provides power to these peripherals...



2. Detailed Component Breakdown

- **Processor / CPU (Central Processing Unit):** Acts as the central processing unit of the PLC. It monitors and processes the information contained in the data registers and executes the user-entered Ladder Logic program, and writes the calculated results into the output modules. The CPU constantly repeats the Input Scan → Logic Execution → Output Scan cycle.
- **Memory:** Divided into **Program Memory** and **Data Memory**. Program Memory stores the application compiled logic (e.g., ladder diagrams). Data Memory holds I/O status tables, variable values (BOOL, INT, REAL), timer/counter accumulation, and system registers.
- **Power Supply Module:** Converts high-voltage AC or main DC power (e.g., 120/230V AC or 24V DC) into regulated low-voltage DC power required to run the CPU, backplane bus, and internal module electronics.
- **Input Module:** Acts as the interface between field sensing devices and the CPU. It receives physical electrical signals from switches or sensors, provides optical isolation to protect sensitive electronics, and converts the signals into digital logic data (0 or 1) for the processor.
- **Output Module:** Serves as the interface between the CPU and field load devices. It converts low-power logic instructions from the CPU into higher-voltage/current control signals to drive relays, motor starters, valves, or indicator lights.
- **Rack / Chassis:** The physical frame and printed circuit backplane that mechanically holds and electrically interconnects the CPU, power supply, and I/O modules. It includes the high-speed data bus for inter-module communication.
- **Programming Interface / Device:** An external computer or software suite (such as **OpenPLC Editor**) used by technicians to write, compile, debug, and upload programs to the PLC memory. Once uploaded, the PLC executes the logic autonomously.

Evidence 3 — Input/Output Classification

1. Device Classification Table

#	Device Name	Device Category	Signal Type	Industrial Process Application
1	Liquid Level Float Switch	Input	Discrete (Digital)	Detects when a fluid level reaches a maximum threshold in a storage tank.
2	Normally Open Push Button (Start)	Input	Discrete (Digital)	Operator input to initiate equipment startup.
3	RTD Temperature Sensor	Input	Analogue	Continuously measures process vessel temperatures across a 0°C to 150°C scale.
4	Motor Starter / Contactor	Output / Load	Discrete (Digital)	Switches electrical power to start or stop a heavy industrial conveyor motor.
5	Solenoid Valve	Output / Load	Discrete (Digital)	Electromechanically opens or closes a pneumatic valve to direct fluid flow.
6	Variable Frequency Drive (VFD)	Output / Load	Analogue	Adjusts pump motor speed dynamically via a 4--20 mA control signal.

2. Discrete (Digital) vs. Analogue I/O Explanation

- Discrete (Digital) I/O:** Operates on a binary state system. The signal is restricted to one of two states at any given moment: **ON** (1 / High Voltage) or **OFF** (0 / Low Voltage). *Example:* A push button or float switch—it is either open (0) or closed (1).

- **Analogue I/O:** Operates over a continuous range of varying electrical parameters (such as 0--10V DC or 4--20 mA) over time. It represents physical quantities that fluctuate continuously. *Example:* A pressure transmitter or temperature sensor reporting exact, fluctuating values (e.g., 24.2°C, 24.5°C).

Evidence 4 — OpenPLC and Ladder Logic Practical

1. Screenshot 2 — OpenPLC Motor_Control Diagram

The screenshot displays the ICDFA-PLC-Engineering-v1.0 software interface, which is running within an Oracle VirtualBox. The interface is divided into several panels:

- Project Panel (Left):** Shows a tree view with 'Ladder Diagram' expanded, containing 'Motor_Control' and 'Res0'.
- Variable Declaration Table (Top Center):**

#	Name	Class	Type	Location	Initial Value	Option	Documentation
2	Start	Local	BOOL				Start push button input
3	MotorOutp	Local	BOOL				Output coil to motor
- Ladder Logic Diagram (Center):** Shows a single rungs with two parallel normally open contacts labeled 'Stop' and 'Start', followed by a coil labeled 'MotorOutput'.
- Debugger Panel (Right):** Includes a 'Library' tab, a 'Range' dropdown set to '1s', and a variable declaration for 'Config0.Res0.instance0.*'. Below this is a variable declaration for 'Stop' with a range from 0 to 1000.
- Console/PLC Log Panel (Bottom):** Contains tabs for 'Search', 'Console', and 'PLC Log'.

Caption: Figure 4.1 — OpenPLC Editor displaying the Motor_Control ladder diagram showing the variable table, two power rails, contacts, and output coil.

2. Variable Table & Element Mapping

Variable Name	Data Type	Ladder Contact / Coil Symbol	Physical Function
Stop	BOOL	Normally Closed Contact] [	Stop push button input. Allows rung continuity when unpressed (TRUE).
Start	BOOL	Normally Open Contact] [	Start push button input. Blocks continuity until pressed (TRUE).
MotorOutput	BOOL	Parallel NO Contact] [	Latching / Seal-in contact bound to output state.
MotorOutput	BOOL	Output Coil ()	Controls physical motor contactor/relay load.

3. Detailed Explanation of Control Logic Execution

A. Circuit Topology

The logic consists of a single ladder rung connected between the **Left Power Rail** (24V DC) and the **Right Power Rail** (0V/Neutral):

1. The **Normally Closed Stop contact** is placed in series with the entire circuit on the left.
2. The **Normally Open Start contact** and the **Normally Open MotorOutput latch contact** are connected in parallel with each other.

3. The **MotorOutput coil** is connected on the far right end of the rung.

B. Step-by-Step Control Flow

1. **Idle State (Motor Stopped):** The Stop push button is unpressed. Because it uses a Normally Closed () / () contact, it evaluates as **TRUE (1)**, allowing power to pass through. The Start button is unpressed (Normally Open) (), evaluating as **FALSE (0)**. The MotorOutput coil is currently de-energized, so the parallel MotorOutput contact is open (FALSE). **Result:** No complete path exists from left rail to right rail. MotorOutput remains **OFF**.
2. **Starting the Motor (Button Pressed):** The operator presses the momentary Start button. Start switches from FALSE to **TRUE (1)**. Current flows from the Left Rail → through Stop (TRUE) → through Start (TRUE) → to MotorOutput coil. **Result:** MotorOutput coil energizes to **TRUE (1)**, turning ON the motor.
3. **Latching / Seal-in Action (Button Released):** As soon as MotorOutput energizes, its associated parallel contact MotorOutput] [closes (**TRUE**). When the operator releases the momentary Start button, Start opens back to **FALSE**. Power continues to flow through Stop (TRUE) and through the closed **MotorOutput seal-in contact (TRUE)**, keeping the coil energized. **Result:** The motor stays running continuously even after the Start button is released.
4. **Stopping the Motor (Stop Pressed):** The operator presses the Stop button. The Normally Closed Stop contact opens, evaluating to **FALSE (0)**. Path continuity across the entire rung is broken. **Result:** MotorOutput coil de-energizes (**FALSE**). The seal-in contact MotorOutput opens back to **FALSE**. When Stop is released, the circuit returns to the idle state.

Evidence 5 — Knowledge Check & Cybersecurity Reflection

1. Knowledge Check Responses

1. **What is the difference between an input module and an output module?**

Response: An input module is used to receive external program triggering signals from field sensing devices (e.g., push buttons, sensors) and convert them into digital logic for the PLC CPU. An output module is used to receive control logic

commands from the CPU and convert them into power signals for operating field load devices (e.g., motors, solenoid valves).

2. **Why is a normally closed contact commonly associated with stop or emergency-stop logic?**

Response: Normally Closed (NC) contacts provide **failsafe operation**. Under normal operating conditions, current flows continuously through the closed contact. If a control wire breaks or power drops, the path automatically opens (FALSE), immediately halting equipment to prevent unsafe conditions.

3. **Give one example of a discrete I/O signal and one example of an analog I/O signal.**

Response:

- **Discrete I/O:** A liquid level switch returning 1 (ON) when full and 0 (OFF) when empty.
- **Analogue I/O:** A 4--20 mA signal from a VFD controlling motor speed between 0 and 1800 RPM.

4. **What operational risk can arise when industrial automation devices become network-connected?**

Response: Network connectivity exposes PLCs to cyber attacks, remote unauthorized access, and command injection. Attackers can tamper with ladder logic or spoof I/O telemetry, causing physical hardware damage, operational downtime, or severe safety hazards.

2. Cybersecurity Reflection: Network-Connected PLCs

Connecting Programmable Logic Controllers (PLCs) to industrial IP networks delivers substantial operational advantages, including centralized monitoring, remote diagnostics, real-time telemetry, and streamlined synchronization across plant operations. Supervisors can dynamically adjust production schedules, analyze field sensor trends, and perform firmware or program updates without needing physical access to control cabinets. This interconnectivity drastically reduces operational downtime and increases overall manufacturing efficiency.

However, transitioning PLCs from air-gapped relay replacements to networked endpoints exposes critical infrastructure to serious cybersecurity risks. Legacy industrial protocols like Modbus and Ethernet/IP often lack native authentication, encryption, or integrity checks, allowing malicious actors who gain network access to perform unauthorized command

injection, alter ladder logic, or spoof I/O telemetry. Such cyber-attacks can cause severe operational disruption, physical equipment damage, environmental hazards, or catastrophic safety risks in facilities like water treatment or power generation plants. Therefore, robust network segmentation, industrial firewalls, deep packet inspection, and rigorous access control frameworks are vital to safeguarding networked Operational Technology (OT) environments.